

QWeave: Correctness-First Deterministic and Learned Qubit Routing

QWeave Research Team

21 September 2026

Abstract

Limited two-qubit connectivity requires a compiler to assign logical qubits to physical sites and route nonlocal interactions. These choices can add SWAPs and circuit depth, while an apparently low-cost circuit is useful only if it preserves the intended computation. This paper develops a correctness-first evaluation of QWeave's deterministic and learned compiler paths. The deterministic path combines a weighted interaction-graph mapper, strict local improvement, and shortest-path SWAP routing; identity mapping, Qiskit SABRE, and a small exact initial-mapping oracle provide separate reference points. Structural replay is supplemented by exact small-unitary checks and deterministic statevector probes for wider circuits and idle physical sites. A fixed-route scheduler and directed-gate lowering pass measure timing effects separately from mapping and routing. The learned path combines a guarded Gymnasium environment, masked graph actor-critic, and PPO with deterministic fallback. Benchmark v2 freezes 180 synthetic cases before evaluation, including 36 held-out QFT and hardware-efficient test cases at four, six, and eight logical qubits. Ten policies are trained for 4,096 steps across five seeds, producing 792 test records. Every tested output passed its applicable numerical semantic check, and every direction-lowered cost circuit passed ordered-coupler legality. The paired GNN-PPO minus weighted depth difference was 0 gates with a 95% bootstrap interval of [0,0]. GNN-PPO and no-message-passing PPO also had paired median depth and SWAP differences of 0 with intervals [0,0]. Training strongly improved on the untrained policy, but the study does not show a message-passing or learned-routing advantage over the deterministic references.

Introduction

Near-term devices constrain which physical qubit pairs can perform two-qubit operations. A logical circuit that ignores this graph requires a physical assignment and, where necessary, SWAP insertion before its gates can execute. Layout and routing are related but distinct compilation stages [5]. SABRE uses bidirectional heuristic search to improve layout and SWAP selection [1]. It is therefore an essential baseline, rather than a straw-man comparator.

QWeave currently supplies a deterministic reference path that makes the mapping convention, tie-breaking, layout updates, and metric extraction explicit. The research question is whether its weighted static mapping surrogate or guarded learned router helps on defined circuit families once the routed circuits are checked for legality and semantics. The scheduler is measured on each fixed legal route, while learned routing is evaluated through separate message-passing and training controls. Existing reinforcement-learning and graph-guided routing studies [2, 3] already establish that learning for this problem is not a novelty claim by itself. This manuscript's contribution is a precise implementation account, an auditable comparative protocol, and a bounded negative result on the tested message-passing and learned-routing advantage, not a claim of optimality.

Related Work

SABRE combines an initial layout search and SWAP-based routing, including forward-backward circuit traversal [1]. In Qiskit, layout and routing may both change the circuit and add idle physical sites [6]. Pozzi et al. studied deep reinforcement learning for qubit routing with depth overhead as an objective [2]. Sinha et al. combined a graph neural network with Monte Carlo tree search, rather than PPO, for depth-oriented routing [3]. Nannicini et al. formulated joint assignment and routing through integer programming on small instances [4]. QWeave's CP-SAT component has a narrower role: it optimizes only the

static initial-mapping surrogate. It cannot certify optimal SWAP count or compiled depth.

Problem and Deterministic Method

Let C be an ordered circuit on logical qubits $L=0,\dots,n-1$ and $H=(P,EH)$ an undirected hardware graph. The initial injective mapping $m:L \rightarrow P$ uses the convention $m(\text{logical})=\text{physical}$. The interaction graph GC contains all logical qubits, including isolates; an edge (i,j) has weight w_{ij} equal to its two-qubit interaction count in the default experiment. The static surrogate is

$$J(m) = \text{sum over } (i,j) \text{ in } E_C \text{ of } w_{ij} d_H(m(i),m(j)). \quad (1)$$

where d_H is shortest-path distance. An optional decay weights an interaction by γ^t for its circuit-operation index t , but the present smoke comparison uses integer counts without decay.

The mapper seeds a high weighted-degree logical qubit on a high-centrality physical node. It then assigns remaining logical qubits greedily according to interaction weight with already mapped neighbours and partial distance cost. Deterministic order resolves ties. Pairwise local search exchanges assigned physical sites, selects the best pair, and accepts a move only if J strictly decreases. Since the set of injective assignments is finite, strict descent terminates; the implementation also caps iterations.

For each source operation in order, the fallback router finds a deterministically chosen shortest hardware path if its two mapped qubits are nonadjacent. It moves the first logical qubit toward the second by edge-local SWAPs, updates both the forward and inverse layout after every inserted SWAP, and emits the operation when adjacent. The final layout is returned alongside the physical circuit. Static distance in Equation (1) is a heuristic for future routing cost: a lower value does not imply fewer SWAPs, minimum legal depth, or higher physical fidelity.

The deterministic Qiskit adapter accepts unconditioned one- and two-qubit gates, one-qubit resets, and a terminal measurement suffix while preserving global phase and classical destinations. It rejects barriers, higher-arity gates, dynamic control flow, and classically conditioned operations rather than silently passing unsupported semantics through the router. A trace with one event per emitted operation is replayed to check source operation order, parameters, quantum and classical targets, inserted SWAP layouts, and the final mapping. SABRE receives both orientations of each undirected routing edge so the paths use the same adjacency interpretation. Equal-width unitary circuits of at most six qubits use an exact layout-aware operator check; wider circuits and outputs with idle physical sites use deterministic layout-aware statevector probes up to 12 physical qubits. Reset and terminal measurement semantics remain structurally, rather than numerically, assessed.

The Basic reference uses identity mapping with this same router. The SABRE reference uses Qiskit transpilation at optimization level zero and a stated seed. CP-SAT minimizes an integer-scaled version of Equation (1) for at most eight logical qubits; only an OPTIMAL solver status certifies that scaled mapping objective. These comparisons isolate the layout choice from the shared deterministic router, while SABRE is a separate compiler path.

Post-Routing Scheduling

The implemented list scheduler accepts an already legal physical circuit. It constructs a dependency graph by preserving operation order on each physical qubit. For operation i with positive duration d_i , the remaining critical-path rank and earliest start are

$$r_i = d_i + \max_{\text{over successors } j \text{ of } r_j}; \quad s_i = \max_{\text{over predecessors } j \text{ of } (s_j + d_j)}. \quad (2)$$

with an empty maximum defined as zero. Ready operations are prioritised by r_i , with source order resolving ties; disjoint work can share a start time. The output preserves every physical wire's gate order, global phase, and operation multiset. It does not insert SWAPs or use unproved gate commutations. Therefore its effect is measured as scheduled depth minus routed depth on the same circuit, never as a comparison between two different mappings. Under unit durations, Qiskit's circuit depth already reflects this dependency parallelism, so a zero depth change is expected. Positive per-gate durations additionally yield a timed makespan but do not constitute a calibrated hardware result.

Routing methods use a shared undirected adjacency graph. Hardware-cost evaluation then lowers each route to the declared directed couplers. A reverse CNOT is implemented by Hadamard conjugation around the available forward CNOT; a SWAP is decomposed into three CNOTs and each direction is lowered independently. Symmetric CZ, CP, and RZZ gates may use either coupler orientation. Scheduling and independent-error proxy costs are computed only after the lowered circuit passes ordered-arc legality. Pre-lowering legality, added operations, and added depth remain separate metrics. The durations and edge errors are declared synthetic assumptions, not device calibration.

Guarded Learned Router

The learned path is isolated from the deterministic compiler. Its Gymnasium environment exposes an Execute action and one SWAP action for each sorted hardware edge. The observation contains five features per physical node, four features per hardware edge, a five-value frontier description, the current logical-to-physical mapping, and a binary legal-action mask. Illegal actions do not mutate the circuit or layout. If the policy repeatedly fails to make progress, or reaches the episode limit, the environment routes the current operation with the validated deterministic shortest-path method. Every completed episode is replay-validated before a routing result is returned. This guard ensures termination and legality on connected undirected hardware with contiguous labels, but may hide a weak policy behind fallback; fallback counts are therefore reported.

The actor-critic encodes node features and performs two rounds of edge-conditioned message passing. It scores every hardware-edge SWAP and the Execute action after applying the legal-action mask. PPO uses clipped policy ratios, generalized advantage estimation, value and entropy terms, and gradient clipping. We train five explicit seeds. A zero-message-passing PPO control retains the optimizer and reward, while an untrained graph control retains the architecture without PPO updates. All learned variants begin with the weighted deterministic initial mapping, which makes this a routing-policy comparison rather than a learned-layout experiment.

Validation and Experimental Design

The structural validator checks mapping completeness, injectivity, source gate replay, inserted SWAP consistency, global phase, and undirected edge legality for every two-qubit emitted operation. Edge legality alone cannot establish that the logical unitary was preserved. For equal-width small unitary circuits, the test oracle constructs the permutations induced by initial and final layouts and checks

$$U_{\text{routed}} P_{\text{initial}} = P_{\text{final}} U_C. \quad (3)$$

The exact check covers equal-width circuits of at most six qubits. Wider unitary cases and circuits with idle physical sites use reproducible layout-aware statevector probes up to 12 physical qubits. Tests include mixed one- and two-qubit gates, source SWAPs, wider layouts, idle sites, and deliberate trace/layout corruption. Reset and terminal-measurement circuits use structural replay; branch-dependent dynamic control remains an explicit unsupported case.

Benchmark version 2.0 freezes 180 cases at four, six, and eight logical qubits, two instances per family, before evaluation. Sixty training cases use GHZ, layered-CX, seeded-interaction, Bernstein-Vazirani, and QAOA MaxCut families on path and ring hardware. Eighty-four validation cases cover topology transfer and held-out families. The 36 untouched test cases combine held-out QFT and hardware-efficient families with two-row grids, chorded rings, and larger grids containing idle physical sites. Every case stores QASM, source hash, undirected and directed edges, and declared synthetic edge-error values; the manifest hash is fixed before training.

Three candidates receive 1,024 training steps and validation seed 101. The selection score is median depth plus half the SWAP count plus five times the fallback count. The selected candidate is then trained for 4,096 steps at seeds 7, 17, 29, 41,

and 53 for both GNN-PPO and the no-message control. The untrained control uses the same five initialisation seeds. SABRE uses the same five seeds; Basic and weighted routing are deterministic. Runtime is the median of three measurements after one warmup. Seeds are collapsed within each case before paired differences are formed. We report family quartiles, validity, fallbacks, runtime distributions, counterexamples, and deterministic 95% bootstrap intervals from 5,000 resamples with seed 20260921.

Results and Limitations

A fresh local run passed 69 tests under Python 3.11.15 and Qiskit 2.5.2. The aggressive validation candidate scored 19.5; conservative and standard each scored 24.0. Table 1 reports the 36-case held-out test aggregate. For seeded methods, each case contributes its within-case seed median before aggregation. All 792 outputs passed their applicable numerical semantic check. QWeave routes additionally passed symbolic replay; SABRE has no corresponding QWeave trace. Every unit-duration scheduling delta was zero.

Table 1 Benchmark-v2 test summary. Runtime is median wall time in seconds.

Method	Depth	SWAP	Runtime	Fallback
Basic	48.5	11.5	0.0022	0
Weighted	36.0	0.0	0.0036	0
SABRE	46.0	7.0	0.0028	0
GNN-PPO	37.5	0.0	0.3473	4543
No-message PPO	36.0	0.0	0.1040	742
Untrained GNN	198.0	193.0	0.6681	8059

The paired GNN-PPO minus weighted depth difference was 0 gates with interval [0,0]; versus SABRE it was 0 with interval [0,5]. Its paired SWAP difference versus weighted routing was 0 with interval [0,5]. GNN-PPO and no-message PPO had paired median depth and SWAP differences of 0 with intervals [0,0], providing no evidence that message passing helped. Training did matter relative to the untrained GNN: the paired GNN-PPO depth difference was -162.5 with interval [-184,-106], and the SWAP difference was -175 with interval [-223,-148].

Family results expose the aggregate mechanism. On hardware-efficient circuits, weighted, GNN-PPO, and no-

message PPO all had median depth 27 and no SWAPs. On QFT, weighted routing had median depth 71.5 and 16.5 SWAPs, GNN-PPO had 83.5 and 41, and no-message PPO had 90 and 31.5. GNN-PPO improved by eight depth layers on qft-i1-6q-chorded-ring, but regressed by 89 layers and 130 SWAPs on qft-i1-8q-grid2. The GNN policy invoked 4,543 test fallbacks, compared with 742 for no-message PPO and zero for deterministic methods. Seed 29 was unstable, ending with 212 training fallbacks and a recent mean return near -103.

QWeave benchmark v2 learning curves

Median recent episode return across five seeds; shaded band is the interquartile range

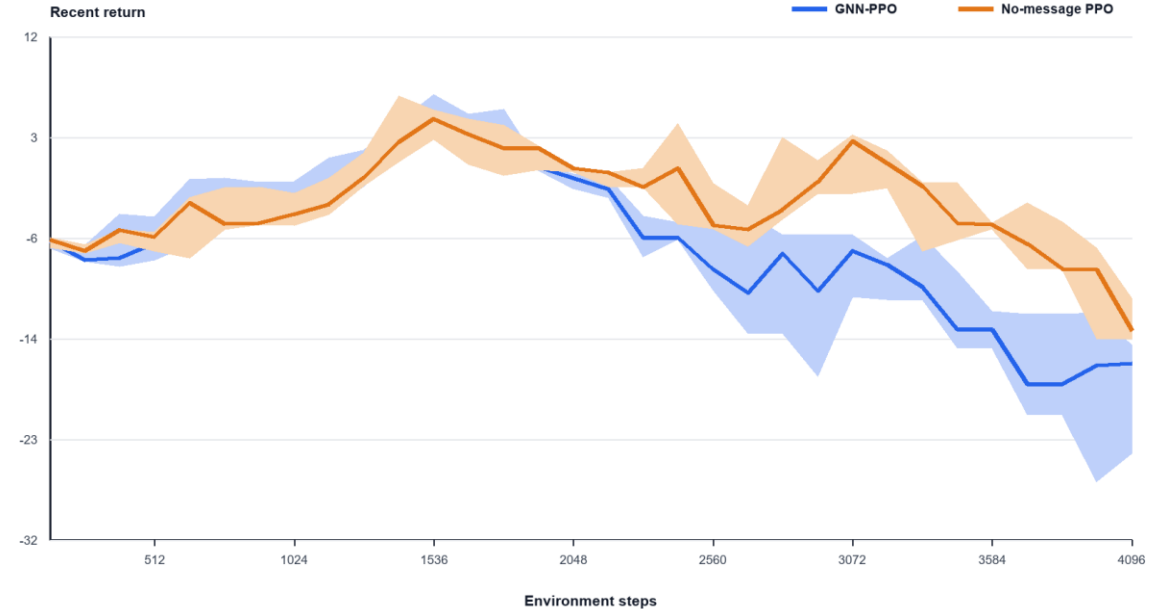


Figure 1 Five-seed learning curves. Lines show the median recent episode return and bands show the interquartile range.

Before direction lowering, most routed outputs were illegal on the one-way couplers. After reverse-CNOT and SWAP synthesis, all 792 cost circuits passed ordered-arc legality. Median added depth was 32 for weighted routing, 34 for SABRE, 37.5 for GNN-PPO, 28.5 for no-message PPO, 57 for

Basic, and 441 for the untrained GNN. The corrected paired GNN-PPO minus weighted makespan difference was 0 ns with interval [0,1113]. These proxy costs therefore do not provide a learned advantage. The timings and error probabilities use declared synthetic values and are not device-calibration results.

The original completed record is retained because audit found that its duration fields scheduled pre-lowering circuits on an undirected graph. A schema-2.1 replay used the saved checkpoints and frozen cases to recompute only direction-dependent cost fields. Route depth, SWAP count, runtime, policy, split, and semantic results did not change. This correction prevents the directed-legality check from being presented separately from an incompatible undirected cost calculation.

Conclusion

QWeave now combines an explicit deterministic reference, a small mapping oracle, fixed-route scheduling, and a guarded learned router with auditable ablation controls. Benchmark v2 extends semantic assessment to wider and idle-site unitary circuits, retains dynamic-control limits explicitly, and evaluates declared directed-coupler costs only after legal lowering. The results show a clear benefit of PPO training over an untrained policy, but no benefit from the tested message-passing component and no learned advantage over weighted routing. The finite synthetic suite does not establish optimal routing, calibrated fidelity, production readiness, or state-of-the-art performance. Further work should target policy stability, reduce fallback dependence, and preregister larger device-derived circuit and calibration suites before making a superiority claim.

References

- [1] G. Li, Y. Ding, and Y. Xie. Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices. ASPLOS, 2019. arXiv:1809.02573.
- [2] M. G. Pozzi, S. J. Herbert, A. Sengupta, and R. D. Mullins. Using Reinforcement Learning to Perform Qubit Routing in Quantum Compilers. arXiv:2007.15957, 2020.
- [3] A. Sinha, U. Azad, and H. Singh. Qubit Routing using Graph Neural Network aided Monte Carlo Tree Search. AAAI, 2022. arXiv:2104.01992.
- [4] G. Nannicini, L. S. Bishop, O. Gunluk, and P. Jurcevic. Optimal Qubit Assignment and Routing via Integer Programming. ACM Transactions on Quantum Computing, 2022.
- [5] IBM Quantum. Transpiler stages. IBM Quantum Documentation, 2026.
- [6] IBM Quantum. SabreLayout API reference. IBM Quantum Documentation, 2026.